

Resumo: Simulação e Física no Pygame

Simulação e Física no Pygame

Este documento apresenta um resumo detalhado dos conteúdos abordados na pasta `aulas/pygame/fisica/`, focados na implementação de movimento e física newtoniana básica aplicados à criação de jogos com Pygame.

1. Introdução à Simulação Física (`index.md`)

Todo jogo necessita de regras que definem o funcionamento de seu universo. Muitas dessas regras encontram inspiração na física do mundo real para trazer precisão e previsibilidade aos movimentos dos elementos na tela. Nesta unidade, o foco recai sobre dois modelos clássicos da física newtoniana: - **MRU (Movimento Retilíneo Uniforme)**: Movimentos sem aceleração (velocidade constante). - **MRUV (Movimento Retilíneo Uniformemente Variado)**: Movimentos com aceleração (velocidade varia de forma constante).

2. Lidando com o Tempo e Game Loop (`tempo.md`)

Ao projetar o movimento de um personagem, sabemos quantos pixels ele deve percorrer em determinado número de segundos (velocidade). Porém, **a taxa de atualização (framerate) dos jogos varia** de acordo com a capacidade do hardware ou da carga de processamento exigida em cada frame.

Portanto, **não podemos apenas somar um valor de deslocamento fixo por iteração do game loop** (como exemplificado pelo script inicial `simulacao.py`, que inocentemente adiciona `0.1` constante no eixo X por loop, amarrando a física à performance do PC). Para manter os movimentos consistentes em relação ao passar do mundo real, a simulação passa a depender da aferição exata do Δt (tempo transcorrido entre frames sucessivos).

3. Movimento Retilíneo Uniforme - MRU (`mru.md`)

No MRU, o objeto se desloca a uma velocidade constante. A ausência de aceleração significa que, em um intervalo Δt , o corpo avança em proporção linear.

Para implementar isso corretamente com o Pygame:

a) O Cálculo do Tempo (Δt) e FPS

Para saber o Δt (a diferença exata de milissegundos entre o frame anterior e o atual), fazemos uso da função nativa do Pygame: 1. `pygame.time.get_ticks()`: Retorna os milissegundos passados desde que `pygame.init()` foi chamada. 2. Armazenamos ao final do update num campo de estado (`last_updated`), e calculamos o Δt do novo frame simplesmente através de `tempo_atual - last_updated`. O ideal é dividir por 1000 para se obter o Δt na escala de *segundos*. 3. O monitoramento de desempenho se dá pelos **Frames por Segundo (FPS)**. Sua conversão acontece pela fórmula pura: $FPS = \frac{1000}{\Delta t_{ms}}$.

b) Atualizando a Posição

Dentro da função encarregada de progredir os componentes (ex: `atualiza_estado`), aplica-se a lei do MRU:

```
prox_posicao = posicao_atual + (velocidade * delta_t_em_segundos)
```

Como resultado, se o framerate de um jogador despencar durante o jogo, o Δt dele aumentará e multiplicará a velocidade da mesma forma que para um framerate alto o Δt abaixa e minimiza as progressões: o resultado visual da corrida da sua posição se sustenta equivalente independente de perdas de hardware.

c) Colisões Básicas com Retornos

No Pygame lidamos com planos de duas dimensões, logo a física e as colisões contra as áreas restritas necessitam atuar no eixo 1D iterativamente para a componente X e a componente Y. - Para bater e efetivamente refletir na parede da tela, testa-se as fronteiras do objeto (como subtrair ou adicionar o seu **raio** num cenário de objeto redondo) confrontando com $x < 0$ (passou da borda esquerda/topo) ou com o limite de tamanho da janela (passou da borda direita/baixo). - Identificando que foi deflagrada a saída do objeto, deve-se espelhar (ou seja, multiplicar por -1) somente a velocidade referente daquele eixo invertido, seguido do reposicionamento de urgência que retorna a posição para imediatamente o mais fundo da própria tela para impedir que continue escapando do limite.

4. Movimento Retilíneo Uniformemente Variado - MRUV (`mruv.md`)

Pela introdução da taxa constante de alteração de velocidade num intervalo – como é a **Aceleração da Gravidade** –, partimos pro modelo MRUV.

Na matemática aplicada vemos que a fórmula de MRUV necessita preencher por inteiro $S = S_0 + v_0 \cdot t + \frac{a \cdot t^2}{2}$, mas na nossa abordagem contínua através da iteração a cada momento de quadros renderizados, opta-se por calcular de maneira encadeada de passos menores:

Conceito base de Aceleração é a *variação da velocidade*. 1. Atualizamos diretamente como a própria **velocidade varia através do tempo pela aceleração**:

`prox_velocidade = velocidade_atual + (aceleracao * delta_t_em_segundos)`

2. Imediatamente a seguir, **atualiza-se a posição consumindo a nova velocidade ajustada**:

`prox_posicao = posicao_atual + (prox_velocidade * delta_t_em_segundos)`